Project report for an AWS multiplayer matchmaking engine

Alessandro Fantesini[*a,1], Antonia Stieger[† b,2] and Ondrej Müller[‡ c,3]

[a] *Free University of Bozen-Bolzano, Italy*
[b] *Free University of Bozen-Bolzano, Italy*
[c] *Free University of Bozen-Bolzano, Italy*

Abstract—This project studies how a cloud-based backend can support multiplayer game matchmaking. Our main question was: can we build a small but realistic matchmaking engine that authenticates players, groups them by skill, and assigns them to a running game server while keeping the infrastructure mostly serverless and easy to deploy? To answer this, we implemented an AWS system. Players create matchmaking tickets through a REST API, a scheduled matchmaker groups compatible tickets using an Elo-based rule, and each successful group receives the address of the dedicated server running the Red Eclipse game in a Docker container. The project was partly inspired by an online tutorial video [7] about building multiplayer game infrastructure on AWS, but we adapted the idea to our own architecture because of the limits introduced by the AWS Academy Learner Lab. We tested the system with individual end-to-end tests, an eight-player group test, and a benchmark for starting game servers on ECS/EC2. The implementation showed that a serverless design is a good fit for the ticketing and polling part of matchmaking, while the actual game server still needs a persistent server. The main tradeoff we observed is between the matchmaking time spent in the queue and the fairness of the match regarding Elo and latency.

Keywords—*cloud computing, distributed systems, multiplayer games, matchmaking, AWS, serverless*

1. INTRODUCTION

The main idea of our project was to build the backend part of a multiplayer game matchmaking system in the cloud. In many online games, the player does not manually choose a server from a list. Instead, the player presses a button, waits in a queue, and the game backend finds other players with a similar level before sending everyone to a game server. This seems simple from the player's point of view, but it involves several distributed-system problems: authentication, shared state, queue management, fairness, server allocation, scaling, failures, and latency.

The questions we tried to answer were: *how can we implement a cloud-based matchmaking engine that best balances the time players spend in the queue waiting and the fairness of the match? What is the best solution from a performance and price point of view?*

We focused on a session-based game model. A player has a profile with an Elo rating, starts matchmaking through an authenticated API call, and receives a ticket. A background process periodically scans all waiting tickets. If enough players are waiting and their Elo values are close enough, they are grouped into one match. The system then assigns them to a running Red Eclipse game server container and provides the player with the server IP address and UDP port. We did not try to build a complete commercial system. The goal was instead to understand the infrastructure pattern: how a cloud backend can move from "player wants a match" to "player has a server to connect to".

2. RELATED WORK

Matchmaking has two major goals that often conflict: matches should be found quickly, but they should also be good matches. The meaning of "good" depends on the game. For a competitive game it may mean similar skill. For a fast action game it may mean low latency. Amazon GameLift FlexMatch [3, 4] describes this as a rule-based matching problem where developers define attributes such as skill or latency, then relax rules as players wait longer. This idea influenced our implementation: our matchmaker starts with a strict Elo tolerance and then widens the tolerance over time.

League of Legends is a useful example of this tradeoff from an existing game. Riot describes its matchmaking as trying to balance fair matches, position preferences, and quick queues [6]. The exact system is much more complex than ours, but the basic problem is similar: if the system waits too long it becomes frustrating, while if it accepts a match too quickly the result may feel unfair. This helped us frame our own simplified version as a balance between Elo fairness, queue time, and latency. Skill-based matchmaking is strongly connected to rating systems. The classic starting point is Elo's rating system [1] for chess. Elo represents player skill as one number, which makes it very convenient for a small project like ours. We chose a simple Elo value because our main contribution was the cloud infrastructure.

Another important research direction is latency-aware matchmaking. Agarwal and Lorch [2] studied matchmaking for online games and other latency-sensitive peer-to-peer systems using a very large trace from Xbox players. Their work shows that players care strongly about lag and that matchmaking can benefit from latency prediction rather than measuring every possible connection at queue time. Our project did not implement latency-based region selection, but this paper is relevant because it explains a limitation of our design: two players with similar Elo values might still have a bad experience if they are far away from the server or from each other. We also looked at cloud game hosting ideas from AWS. GameLift is the managed AWS service for dedicated session-based game servers, and FlexMatch can be connected to game session queues. We could not use GameLift directly due to the AWS Academy Learner Lab constraints. Instead, we recreated a smaller version with API Gateway, Lambda, DynamoDB, and ECS. The YouTube video [7] we came across during the topic search was useful as a practical inspiration for how cloud services can be combined for game backend workflows. Our final design is simpler than a managed GameLift architecture, but it follows the same broad separation between matchmaking logic and game-server hosting.

3. INFRASTRUCTURE AND APPLICATION DESIGN

The system is deployed with AWS SAM [5], so the infrastructure is defined as code in `template.yaml`. The application has five main parts. First, Amazon Cognito manages users and creates JSON Web Tokens (JWT). Second, API Gateway validates those JWTs and exposes these REST API endpoints:

- `POST /register`: create a Cognito user and a player profile.
- `POST /match`: create a matchmaking ticket for the authenticated player.
- `GET /match/\{ticketId\}`: check whether a ticket is still searching or already matched.
- `DELETE /match/\{ticketId\}`: cancel a searching ticket.
- `GET /profile`: return the authenticated player's profile.

Third, Lambda functions implement the application logic. Fourth, DynamoDB stores player profiles and matchmaking tickets. Finally, ECS on EC2 runs the actual Red Eclipse server containers. Figure 1 shows this architecture as one flow: clients authenticate and call the API, Lambda functions read and update DynamoDB state, and the scheduled poller connects successful matches to ECS game-server containers on EC2.

The `PlayerProfiles` table is keyed by the Cognito user ID and stores username, Elo, wins, and losses. The `MatchmakingTickets` ta-

*alessandro.fantesini@student.unibz.it
†antonia.stieger@student.unibz.it
‡ondrej.mueller@student.unibz.it

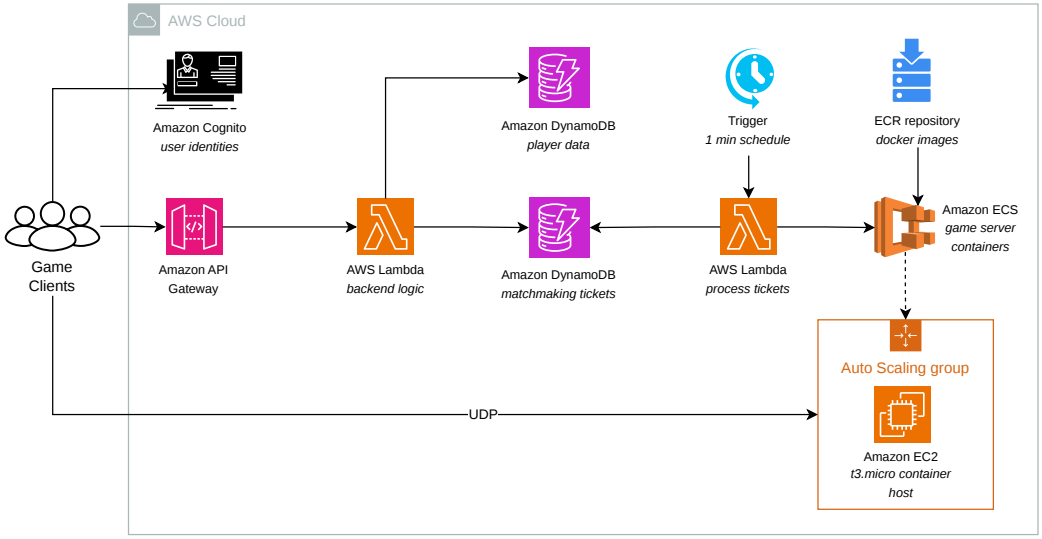

Figure 1. AWS architecture of the matchmaking backend and game-server hosting layer.

ble is keyed by ticket ID and stores the user ID, status, Elo, creation time, matched players, server endpoint, ECS task Amazon Resource Name (ARN), and Time-to-Live (TTL). Ticket statuses move through `SEARCHING`, `SUCCEEDED`, `TIMED_OUT`, or `CANCELLED`. The most important component is the `MatchStatusPoller` Lambda, which is run once per minute. The poller scans all `SEARCHING` tickets, removes tickets older than five minutes by marking them `TIMED_OUT`, sorts active tickets by Elo, and then uses a sliding window to find groups of eight players. The allowed Elo spread starts at 200, increases to 400 after one minute, and increases to 800 after two minutes. This is a simple version of matchmaking rule relaxation. After a valid group is found, the poller allocates a game server. ECS maintains a warm pool of Red Eclipse containers. Each container exposes UDP port 26000 internally, but uses dynamic host port mapping, so Docker assigns a random public UDP port on the EC2 instance. The Lambda function calls ECS and EC2 APIs to find the running task, the host port, and the public IP address of the EC2 instance. This endpoint is then written back into all tickets in the matched group.

4. TECHNOLOGIES AND CHALLENGES

We used DynamoDB because it is serverless, fast for key-value access, and easy to integrate with Lambda. API Gateway and Cognito gave us authentication without building our own login system. ECS, ECR, and EC2 were used for the Red Eclipse server because a dedicated game server is naturally containerized. The frontend is a small static interface, mainly for interacting with the API during demos. One challenge was the difference between matchmaking tickets and real server capacity. It is easy to mark players as matched in DynamoDB, but the match is not useful unless a reachable game server exists. We therefore had to connect the matchmaker to ECS and make sure that a matched group receives a real IP and port. Dynamic port mapping was useful because multiple containers can run on the same small EC2 instance, but it also meant the Lambda function had to inspect ECS task network bindings after allocation. The second challenge was working within small cloud resources. The ECS cluster uses `t3.micro` instances, and the task definition gives each Red Eclipse container 128 MB of memory. This is enough for a prototype, but it forced us to think about how many containers can realistically fit on one instance. Finally, the scheduled poller is simple but not perfect. A one-minute interval means a player may have to wait for that full minute even if enough compatible players are already present. For tests we manually invoke the poller, but in a real system we would probably trigger matching more frequently.

5. EXPERIMENTS

The first tests checked the normal API workflow with one and two users: registering, logging in, creating a matchmaking ticket, polling the ticket status, and cancelling a ticket. The second experiment was the eight-player group test. The setup script creates eight Cognito users and DynamoDB player profiles with known Elo values. The test script authenticates all players, submits eight matchmaking requests, invokes the poller, and checks until all tickets reach a terminal state. The third experiment measured the game-server startup time. For this test we first scaled the Auto Scaling Group down to one EC2 instance. Then the script increased the desired capacity from one to two, waited until the new instance registered in the ECS cluster, started the first Red Eclipse container on it, and finally started a second container on the same instance. The reason for starting two containers was to separate the one-time Docker image pull from the normal container start time.

Lastly, we conducted a capacity experiment comparing different EC2 instance configurations. AWS `t3` instances use a CPU-credit mechanism rather than providing their full CPU capacity continuously. A `t3.micro` instance has two vCPUs and earns CPU credits whenever its utilization remains below a fixed baseline of 10% per vCPU (20% total CPU utilization). The first configuration evaluated was our original deployment using a single `t3.micro` instance hosting seven Red Eclipse containers. CPU measurements suggested that each container consumed approximately 3–5% CPU, resulting in a combined utilization of roughly 21–35%. Since this exceeds the 20% sustained baseline of the instance, CPU credits were continuously depleted. Although the configuration achieved the lowest nominal cost per server, it also introduced the risk that all hosted game sessions would be throttled simultaneously once the remaining credits were consumed.

To evaluate a safer deployment strategy, we considered limiting the same `t3.micro` instance to four containers. Under the same CPU assumptions, total utilization would remain close to the 20% baseline. The trade-off is lower server density and therefore a higher effective cost per hosted game server, but the risk of CPU throttling is substantially reduced. We also examined a `t3.medium` configuration with four containers. The larger instance benefits from a higher baseline CPU allocation and a larger CPU-credit reserve. This allows it to tolerate sustained bursts for longer periods before throttling occurs.

Finally, we compared these burstable-instance configurations with an `m6i.large`. Unlike the `t3` family, the `m6i.large` provides dedicated CPU resources and therefore cannot be throttled due to credit exhaustion. The instance can host a significantly larger number of

Table 1. Interpretation of the CPU-credit test and safer game-server density options.

Configuration	Estimated CPU use	Credit behaviour	Interpretation
`t3.micro`, 7 containers	21–35%	Depletes under load	Fits by memory, but exceeds the sustained CPU baseline and risks throttling.
`t3.micro`, 4 containers	12–20%	Neutral or slowly recovering	Safer prototype limit; it stays close to the baseline when servers are active.
`t3.micro`, 3 containers	9–15%	Accumulates credits	Conservative limit with headroom for short bursts.
`m6i.large`, about 60 containers	Dedicated CPU, no burst credits	No credit depletion model	Better production-style choice: higher monthly cost, but predictable CPU instead of burst throttling.

game-server containers while maintaining predictable performance characteristics. The disadvantage is its substantially higher monthly cost, which is difficult to justify at low player counts. However, as server density increases, the cost per hosted game server becomes competitive with the smaller `t3` configurations while eliminating the credit-depletion failure mode entirely.

6. RESULTS

A player can register, authenticate, create a ticket, and poll it. The matchmaker can group compatible tickets and update them atomically using conditional DynamoDB updates, which prevents overwriting tickets that were cancelled or already changed. In the full-match test, the expected successful result is that both tickets reach `SUCCEEDED` and the API returns the matched players plus the allocated server endpoint. When the eight test players have close enough Elo values, the poller should form exactly one group and assign exactly one ECS server endpoint to all eight tickets.

The server startup benchmark gave concrete timing result. Creating a new EC2 instance took 40 seconds until it was booted and registered in ECS. Starting the first container on the new instance took 37 seconds. This included pulling the Red Eclipse Docker image from ECR. Starting a second container on the same instance took only 4 seconds because the image was already cached locally. This gives two different player experiences. In the normal case, when an instance already exists and the image is cached, a new game server starts in about 4 seconds. In the worst case, when the Auto Scaling Group must create a new instance, the player has to wait about 77 seconds: 40 seconds for EC2 provisioning plus 37 seconds for the first container. Running many game servers on one `t3.micro` can drain CPU credits quickly. Once credits are gone, the instance is limited to its baseline CPU performance, which can cause slower game-server behaviour.

Table 1 shows that if we keep `t3.micro`, we should cap placement at three or four containers per instance instead of seven. For a less budget-constrained deployment, the better answer is to move away from burstable `t3` instances. An `m6i.large` is more expensive, but it provides dedicated CPU capacity and removes the credit-depletion failure mode. In other words, the cheapest configuration has the lowest cost per nominal server, but it is also the least reliable once several games are active at the same time. Overall, the strongest result is that the complete path works: the serverless part can handle the ticket workflow, the poller can create an eight-player match, and ECS can supply a real game-server process. The weakest result is that the system reacts slowly when it has to create completely new compute capacity.

7. COST ANALYSIS

Because one of our guiding questions concerned price, we estimated the monthly cost from the resources declared in `template.yaml`. All figures use `us-east-1` prices and should be read as model-based estimates rather than official quotes. The important result is not the exact cent value, but the cost structure: the API, Lambda, DynamoDB, Cognito, S3, ECR, and CloudWatch parts are billed mainly per request or per stored gigabyte, while the game-server layer has a fixed hourly cost whenever at least one EC2 instance is kept running.

Table 2. Baseline monthly cost with one warm server and no players.

Item	Calculation	Monthly
EC2 `t3.micro`	730 h × $0.0104	$7.59
EBS 30 GB gp3	30 × $0.08	$2.40
`MatchStatusPoller`	43,200 invocations	≈$0
DynamoDB	storage + idle scans	<$0.30
CloudWatch Logs	minimal volume	<$0.50
ECR + S3	image + static files	<$0.10
Total		**≈$10–11**

7.1. Baseline cost

By baseline we mean the cost of leaving the stack deployed with essentially no players connected. This is not zero, because we deliberately keep one game server warm so that the first players to arrive do not have to wait for a new instance to boot. The Auto Scaling Group holds a desired capacity of one, so a single `t3.micro` and its attached EBS volume run continuously, while the serverless components stay within their free tiers. As Table 2 shows, keeping one warm server costs roughly ten dollars per month. We consider this an acceptable price for removing the worst-case start-up delay, measured at about 77 seconds when a new instance must be created, from the common case. If cost mattered more than responsiveness, the baseline could be eliminated entirely by scaling the group to zero or deleting the stack between sessions.

7.2. Cost under load

To see how the cost grows with usage we derive every figure from a small set of base assumptions. A *session* is one player starting matchmaking and following it through to a result. We assume:

- **10,000 sessions per day** — an illustrative busy day, and the only invented number; every cost below scales linearly with it.
- **30 days per month.**
- **10 API requests per session**, made up of one `POST /match`, about eight `GET /match/\{ticketId\}` polls while waiting, and roughly one `DELETE`. The eight polls follow from the system's own timing: the matchmaker runs once a minute, so a ticket waits about thirty seconds on average before it is considered, and the frontend polls every four seconds (`POLL_INTERVAL_MS = 4000`), giving $30/4 \approx 8$.
- **2 ticket writes per session** — one when the ticket is created, one when it is matched.
- **8 players per match**, so $10{,}000/8 \approx 1{,}250$ matches per day.

Every row of Table 3 is then a direct multiplication of these values, with the request and item counts shown in the calculation column so each figure traces back to the assumptions above. The four request-driven rows together come to about twelve dollars per month, which shows how inexpensive the per-request services are even at three

Table 3. Monthly cost under an illustrative load of 10,000 sessions per day. Every count is composed from the base assumptions.

Item	Calculation (from base values)	Monthly
API Gateway	$10{,}000 \times 10 \times 30 = 3$M requests $\times$ \$3.50/M	\$10.50
Lambda	3M requests $\times$ \$0.20/M	≈\$0.60
DynamoDB writes	$10{,}000 \times 2 \times 30 = 600$k writes $\times$ \$1.25/M	≈\$0.75
DynamoDB reads (status polling)	$10{,}000 \times 8 \times 30 = 2.4$M reads $\times$ 0.5 unit $\times$ \$0.25/M	≈\$0.30
DynamoDB reads (poller scan)	grows with table size (see Sec. ??)	\$3 to \$50+
Cognito	10,000 MAU, below the 50,000 free tier	\$0
Data transfer out (game traffic)	grows with match duration (see Sec. ??)	\$10 to \$150+

million requests. The last two rows are left as ranges because they do not depend on request volume: the scan cost grows with the number of tickets in the table, and the egress cost grows with how long matches run. These two are the only parts of the system that can grow into a large bill. For the way we actually use the system — deploying, running a test, and then removing the stack — real usage is a small fraction of this scenario, and the total cost amounts to no more than a few dollars, comfortably within the project budget.

7.3. Cost risks at larger scale

Three aspects of the design behave well as a prototype but would become expensive under sustained production load. First, the matchmaker reads the queue with a DynamoDB `Scan` over the whole `MatchmakingTickets` table. DynamoDB charges for every item the scan reads, not only those that match the `SEARCHING` filter, so the cost grows with the total table size rather than with the number of players queued. Because expired tickets are removed by a background process, finished tickets accumulate and inflate the scan, from a few dollars per month at a few hundred rows to over fifty dollars at ten thousand. A secondary index on the status attribute, queried instead of scanned, removes this cost. The outbound game traffic is billed at \$0.09 per gigabyte beyond the first 100 GB each month and is the largest open-ended cost once real matches are played for any length of time, although the two-instance limit on the Auto Scaling Group bounds it. A related non-monetary risk is that the burstable `t3.micro` depletes its CPU credits under sustained load and is then throttled to its baseline performance, degrading the game server without any explicit cost signal.

8. DISCUSSION

The implementation also shows why even a small matchmaking system becomes complicated. Our matching logic only uses Elo. This is good for demonstrating skill-based grouping, but it ignores network latency, region, party size, game mode, player behaviour, and server health. In our original plan we wanted to balance Elo, queue time, and latency, as shown in Figure 2.

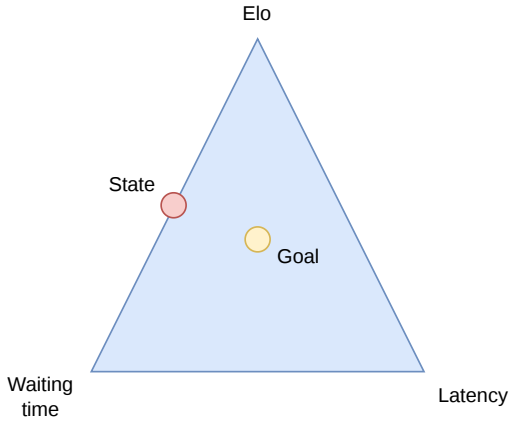

Figure 2. Initial idea for balancing latency, Elo, and waiting time in matchmaking.

However, we could not realistically emulate players connecting from different geographic regions, so we could not measure the real packet round-trip time between remote players and the EC2 game server. Because of this, our final system can form a fair match on paper, but it cannot prove that the selected server offers similar latencies for every player.

The scheduled polling design is another tradeoff. It is easy to implement, cheap, and reliable for a prototype. However, it adds delay and uses DynamoDB scans, which would not scale well to a very large number of tickets. For this course project, the scheduled Lambda was a reasonable compromise because it made the behaviour easy to test and debug. The tests showed that server allocation is the largest practical problem in the current prototype. If capacity is already available, starting another container on a warmed instance took about 4 seconds, which is acceptable for our demo. If a new EC2 instance is needed, the measured wait becomes about 77 seconds. This is too long for a player who has already been told that a match was found. It also shows that matchmaking latency is not only caused by the matching algorithm. The infrastructure state, especially whether an instance is already running or the Docker image is cached, matters a lot.

The `t3.micro` instance is attractive for a student project because it is cheap and fits inside the Learner Lab budget, but the CPU-credit result showed that it is not robust when many containers are placed on the same machine. A simple improvement for our prototype would be to keep fewer game servers per `t3.micro`, for example three or four instead of seven. For a more realistic deployment we would prefer a non-burstable instance such as `m6i.large`: it costs more, but the CPU capacity is predictable and game performance no longer depends on whether the instance has credits left. Another possible improvement would be to keep a second warmed instance during tests, so the image is already cached and the worst measured wait disappears. These are not complicated architectural changes, but they make the cost and capacity tradeoff more visible.

9. CONCLUSION

In this project we designed and implemented an AWS-based multiplayer matchmaking engine. Players authenticate through Cognito, access the system through API Gateway, create matchmaking tickets, and store state in DynamoDB. A scheduled matchmaker groups players by Elo and assigns each completed match to a Red Eclipse server running in ECS on EC2.

The project was successful as a prototype because it demonstrates the complete path from user registration to game-server assignment. It also helped us understand the boundary between serverless infrastructure and persistent game-server infrastructure. The main limitations are the simple Elo-only matching model, the one-minute scheduled polling interval, the slow cold start when a new EC2 instance is needed, and the limited CPU headroom of the `t3.micro` instance. As we did not have access to Amazon Gamelift services, we had to implement the logic ourselves, which was smart from a didactic point of view,

but for a real project it would be the right tool to use as these services are built specifically for matchmaking and game-server allocation. Additionally, if we had more time, we would add real post-match score reporting and automatic Elo updates. Second, we would include latency or region in the matching rules. Finally, we would replace the scheduled scan with a more event-driven or indexed matchmaking design.

REFERENCES

[1] A. E. Elo, *The Rating of Chessplayers, Past and Present*. London: Batsford, 1978, ISBN: 0-7134-1860-5.

[2] S. Agarwal and J. R. Lorch, "Matchmaking for online games and other latency-sensitive peer-to-peer systems", in *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, ACM, 2009, pp. 315–326. DOI: 10.1145/1592568.1592605. [Online]. Available: https://www.columbia.edu/~ebk2141/teaching/csci599-sp13/papers/13_geolocation/matchmaking.pdf.

[3] Amazon Web Services. "Introducing flexmatch, the latest addition to amazon gamelift's matchmaking services". (Aug. 16, 2017), [Online]. Available: https://aws.amazon.com/about-aws/whats-new/2017/08/introducing-flexmatch-the-latest-addition-to-amazon-gamelifts-matchmaking-services/ (visited on 05/27/2026).

[4] A. Qin. "Fine-tune online game matchmaking with amazon gamelift flexmatch rule sets and the amazon gamelift testing toolkit". (Mar. 14, 2024), [Online]. Available: https://aws.amazon.com/blogs/gametech/fine-tune-online-game-matchmaking-with-amazon-gamelift-flexmatch-rule-sets-and-the-amazon-gamelift-testing-toolkit/ (visited on 05/27/2026).

[5] Amazon Web Services. "How aws sam works". (2026), [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam-overview.html (visited on 05/27/2026).

[6] Riot Games. "Matchmaking and autofill". (Mar. 19, 2026), [Online]. Available: https://support-leagueoflegends.riotgames.com/hc/en-us/articles/201752954-Matchmaking-and-Autofill (visited on 05/29/2026).

[7] YouTube. "Tutorial video used as practical inspiration for the cloud game matchmaking project". Referenced by the project group as an implementation idea source. (2026), [Online]. Available: https://www.youtube.com/watch?v=vitoxDcE70o (visited on 05/27/2026).